CSC490 Assignment A3: Pre-training nanochat

Prithvi Seran (1009973343) Vraj Shah (1009102695) Tyseer Toufiq (1009364064)
Joshua Crisologo (1009860438)

Part 1: Architecture Review

The Original nanochat (October 13, 2025)

The original nanochat [1] (commit `3a5e0bc`) is a decoder-only, pre-norm Transformer [2, 3]. Its size is set by the depth L alone: the width is $d = 64L$, each head has 128 dimensions, and the MLP hidden size is $4d$. The d20 model used for the speedrun (Figure 1) has about 561M parameters. Table 1 lists its main design choices.

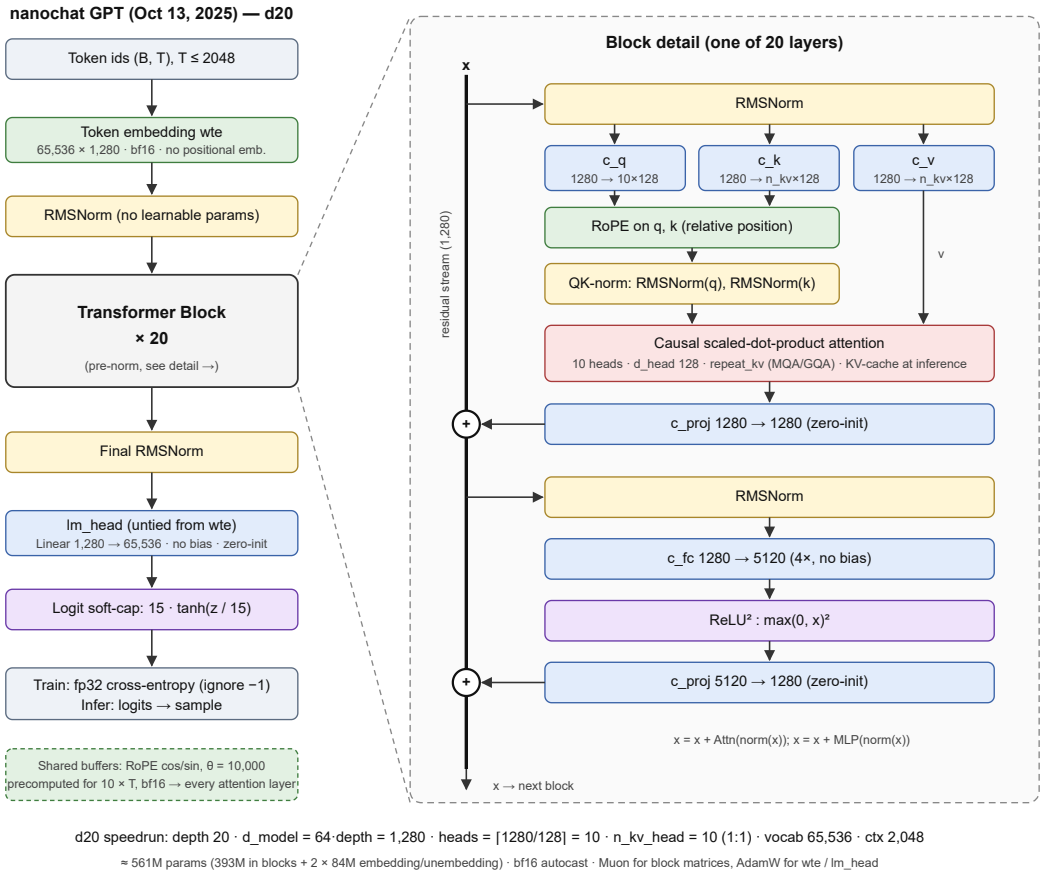


Figure 1: The original nanochat architecture (October 13, 2025) at depth 20. Left: the full model. Right: one of the 20 pre-norm Transformer blocks.

Table 1: Main architecture choices in the original nanochat.

Component	Choice	Why
Position	RoPE on q and k, $\theta = 10{,}000$, all head dims rotated [4]	Relative position with no extra parameters

Component	Choice	Why
Normalization	RMSNorm with no learnable gain, pre-norm, also after the embedding [5]	Cheaper than LayerNorm and keeps the residual stream stable
Attention	Causal multi-head attention with QK-norm [6]	QK-norm bounds attention logits at high learning rates
MLP	$4d$ hidden, ReLU^2 activation [7]	Same cost as GELU and strong in speedruns
Output	Untied `lm_head`; logit soft-cap $15\tanh(z/15)$ [8]	Separate learning rates; prevents over-confident logits
Other	No biases; output projections initialized to zero	Simpler; each block starts as the identity
Optimizer	Muon [9] for the block matrices, AdamW [10] for the embedding and `lm_head`	Muon trains the large matrices faster

Changes in the July 2026 Version

Our baseline is the July 2026 version (`92d63d4`). Most of the changes since October come from the modded-nanogpt speedrun [11]:

- sliding-window attention in an `SSSL` pattern, using FlashAttention-3 [12];
- value embeddings [13] and learned per-layer residual scalars;
- RoPE θ raised to 100,000 and a smaller vocabulary of 32,768 tokens;
- one combined Muon/AdamW optimizer, now with weight decay.

The ReLU^2 MLP has not changed, and RoPE is still applied to every head dimension with one θ for all layers. Our proposed changes target these two parts.

Proposed Changes

ModernBERT [14] updated BERT [15] with components from recent LLMs and tested its choices with ablations. July nanochat already has most of them, including bias-free layers, RoPE, pre-norm, alternating local and global attention, and FlashAttention. We take three ideas from ModernBERT that nanochat lacks: the GeGLU MLP, separate RoPE θ values for local and global layers, and partial RoPE, which ModernBERT tested but did not adopt. Table 2 summarizes them and Figure 2 shows each one.

What we ablate. Part 2 ablates GeGLU and partial RoPE. At d12 neither changes the parameter count or FLOPs, and each is a single flag, so a gap from the baseline that is larger than seed noise can be attributed to the change. We leave change 3 for Part 4: ModernBERT found no difference between θ settings during pretraining and only raised the global θ when it extended its context length. ModernBERT's ablations used encoders of at most 150M parameters, so the results may not carry over to a causal decoder like nanochat.

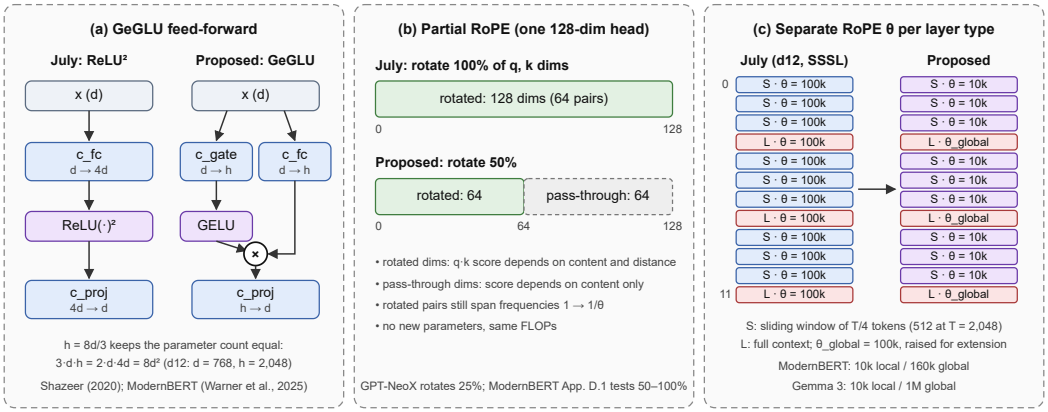


Figure 2: The three proposed changes: (a) GeGLU MLP, (b) partial RoPE, (c) separate RoPE θ for sliding-window (S) and full-context (L) layers.

Table 2: Proposed architecture changes relative to July 2026 nanochat (d = model width, T = context length).

Change	Motivation	Technical Details	Potential Impact
1. GeGLU MLP [16]	In T5, gated MLPs reached lower perplexity than ReLU and GELU MLPs with the same parameter count [16]. PaLM [17], LLaMA [18] and ModernBERT all use one.	$W_{\text{proj}}\big(\text{GELU}(W_{\text{gate}}x) \odot W_{\text{fc}}x\big)$ with $h = 8d/3$, so parameters match ReLU^2 ($h = 2{,}048$ at d12). Flag `--mlp-type=geglu`.	Same parameters and FLOPs. Expect slightly lower loss. ReLU^2 is a strong baseline, so a tie is possible.
2. Partial RoPE [19]	Full RoPE mixes position into every dimension, while unrotated dims can match on content at any distance. GPT-NeoX rotates 25% of each head [19]. ModernBERT found 50–75% slightly better than 100% but kept 100% because its ablations ran at a much smaller scale than the final model.	Rotate the first 64 of the 128 head dims and pass the rest through unchanged. Flag `--rope-frac=0.5`.	No new parameters or FLOPs. Small or no gain at picochat scale; may help context extension.
3. Separate RoPE θ for local and global layers [20]	July uses $\theta = 100{,}000$ everywhere, even in sliding-window layers that only see $T/4$ tokens. ModernBERT uses 10k local and 160k global; Gemma 3 uses 10k and 1M [20].	Two cos/sin tables: sliding-window layers use $\theta = 10{,}000$ and full-context layers keep 100,000. Raise only the global θ when extending context.	No new parameters or FLOPs. We expect no effect at a fixed context length; any benefit should appear during context extension.

Part 2: Ablations on a Nano Nanochat

Results

We tracked every run in the team's Weights & Biases (W&B) project:

`https://forge.coreweave.com/wandb/vraj-shah2121-university-of-toronto/nanochat/workspace`

The three full runs are `d12_baseline`, `d12_geglu` and `d12_rope50`. Each logged validation bits per byte (BPB) every 250 steps, training loss, throughput and MFU every 100 steps, and the CORE score at steps 2000 and 2520. CORE was capped at 500 examples per task. Table 3 summarizes the runs, and Figure 3 compares their training curves.

The W&B configs differ only in the flag under test (`--mlp-type=geglu` or `--rope-frac=0.5`) and the run name. All three runs use seed 42 and 2,520 steps, with the same estimated total of 1.0×10^{18} FLOPs. All three start from a validation BPB of 3.16867, because the zero-initialized output projections make the changes invisible at step 0.

Table 3: Picochat (d12) ablation results. For validation BPB, lower is better; for CORE, higher is better. Gaps are relative to the baseline. Throughput and MFU are medians of the logged steps after the first five.

	Baseline	GeGLU	Partial RoPE (50%)
Hardware	8× H200	8× H200	8× H100
Final validation BPB	0.84537	0.84754	0.84505
Gap to baseline	–	+0.00217	−0.00031
Gap (%)	–	+0.26%	−0.04%
CORE at step 2000	0.1570	0.1547	0.1589
Final CORE (step 2520)	0.1734	0.1656	0.1705
Gap to baseline	–	−0.0078	−0.0029
Tasks better / worse (of 22)	–	9 / 13	8 / 13
Training time (min)	5.23	5.45	5.41
Throughput (million tokens/s)	4.33	4.16	4.10
MFU (%)	41.6	39.9	39.4

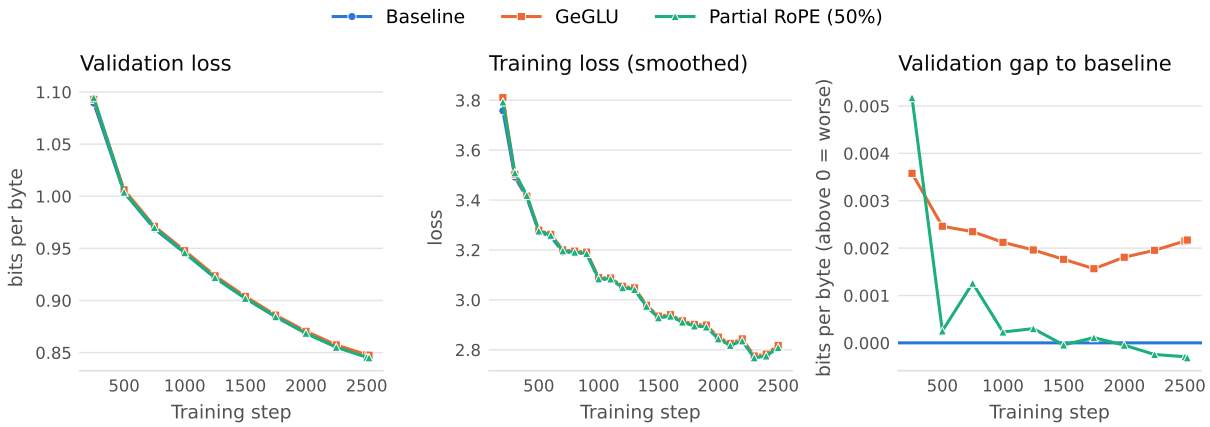


Figure 3: Training curves from the W&B runs. The validation and training curves almost overlap, so the right panel plots each change's validation gap to the baseline. The plots start after the steep early drop (step 250 for validation, step 200 for training loss).

Impact of Each Change

Overall. Neither change clearly improved the model at this scale, which fits the small or null effect we predicted in Part 1. Figure 4 shows why the CORE gaps are weak evidence: individual tasks move by as much as 0.10, in both directions. The standard error of the mean gap across tasks is about 0.008, so GeGLU's gap is 0.95 standard errors from zero and partial RoPE's is 0.34.

GeGLU. GeGLU has a slightly worse validation BPB than the baseline at all 11 evaluations after step 0, with a gap between 0.0016 and 0.0036 that had not closed by the end of training. Its CORE score is also lower, though within the task-level noise above. These evaluations come from one training run and are correlated, so the consistent sign is not 11 independent results. On the same GPU model as the baseline, GeGLU ran about 4% slower in both throughput and training time. Its parameter count and estimated FLOPs match the baseline, so this slowdown is not captured by the FLOP estimate. We do not see the gain reported for gated MLPs in the literature [16], but this run cannot rule one out: it is short and small, and it has only one seed.

Partial RoPE (50%). We cannot distinguish partial RoPE from the baseline. It trails early (a BPB gap of +0.0052 at step 250), catches up by about step 1500, and finishes 0.00031 ahead. It also led on CORE at step 2000 (+0.0019) but trailed at step 2520 (−0.0029), and that change of sign suggests the gap is mostly noise. ModernBERT reported a slight gain at 50–75% [14], and we neither confirm nor contradict that here. We also cannot read its throughput as a cost of the change: this run used H100s while the baseline used H200s, and the change rotates fewer dimensions, so it should not add work.

Limitations. We trained one model per configuration, so we have no direct measure of seed noise. The closest evidence is two identical 100-step smoke tests with the same seed, whose validation BPB differed by 0.00014. That shows only that runs are not fully deterministic, and it is a lower bound on noise, since changing the seed should vary results more. The GeGLU gap is about 15 times that figure and the partial RoPE gap about 2 times. The hardware also differs across runs, as noted above. It should not affect the loss curves much, but it does limit the throughput comparison to baseline versus GeGLU.

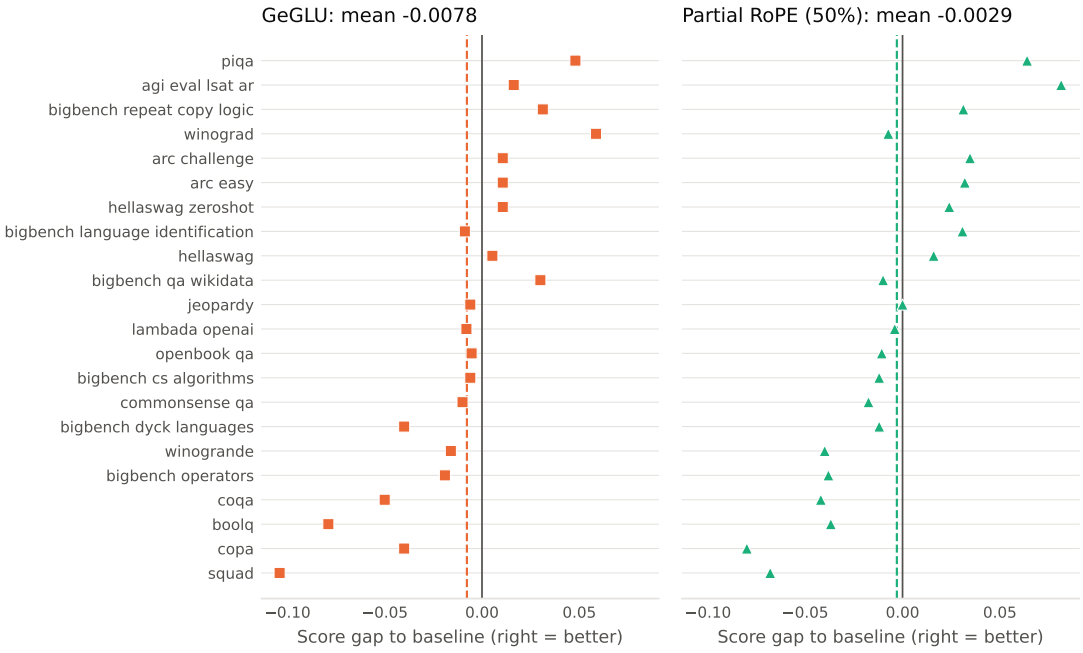


Figure 4: Final CORE gap to the baseline on each of the 22 tasks. The solid line marks zero and the dashed line marks the mean gap. Tasks are sorted by their average gap across the two changes. Both changes gain on some tasks and lose on others.

Part 3: Hardware Efficiency Curves

Part 4: Extending the Context Window

Part 5: Training Our Final Nanochat

References

[1] A. Karpathy, "nanochat: A tiny chatbot arena and training harness," GitHub, 2025. https://github.com/karpathy/nanochat/discussions/481

[2] A. Vaswani et al., "Attention Is All You Need," *NeurIPS*, 2017. arXiv:1706.03762.

[3] A. Radford et al., "Language Models are Unsupervised Multitask Learners," OpenAI technical report, 2019.

[4] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," *Neurocomputing*, vol. 568, 2024. arXiv:2104.09864.

[5] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," *NeurIPS*, 2019. arXiv:1910.07467.

[6] A. Henry et al., "Query-Key Normalization for Transformers," *Findings of EMNLP*, 2020. arXiv:2010.04245.

[7] D. R. So et al., "Primer: Searching for Efficient Transformers for Language Modeling," *NeurIPS*, 2021. arXiv:2109.08668.

[8] Gemma Team, "Gemma 2: Improving Open Language Models at a Practical Size," arXiv:2408.00118, 2024.

[9] K. Jordan et al., "Muon: An optimizer for hidden layers in neural networks," 2024. https://kellerjordan.github.io/posts/muon/

[10] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *ICLR*, 2019. arXiv:1711.05101.

[11] K. Jordan et al., "modded-nanogpt: Speedrunning the NanoGPT baseline," GitHub, 2024. https://github.com/KellerJordan/modded-nanogpt

[12] J. Shah et al., "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision," arXiv:2407.08608, 2024.

[13] Z. Zhou et al., "Value Residual Learning For Alleviating Attention Concentration In Transformers," arXiv:2410.17897, 2024.

[14] B. Warner et al., "Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference," *ACL*, pp. 2526–2547, 2025. https://aclanthology.org/2025.acl-long.127/

[15] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," arXiv:1810.04805, 2018.

[16] N. Shazeer, "GLU Variants Improve Transformer," arXiv:2002.05202, 2020.

[17] A. Chowdhery et al., "PaLM: Scaling Language Modeling with Pathways," *JMLR*, vol. 24, 2023. arXiv:2204.02311.

[18] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv:2302.13971, 2023.

[19] S. Black et al., "GPT-NeoX-20B: An Open-Source Autoregressive Language Model," *BigScience Workshop on Challenges & Perspectives in Creating Large Language Models*, 2022. arXiv:2204.06745.

[20] Gemma Team, "Gemma 3 Technical Report," arXiv:2503.19786, 2025.